

텍스트 기반 RPG 보고서

1. 프로그램 개요

프로젝트의 주제는 텍스트 기반 턴제 RPG 전투 시뮬레이터이다. 플레이어가 직접 캐릭터를 생성하고 몬스터와 전투를 벌이는 구조에서 클래스 설계, 다형성, 파일 입출력 등 C++ 핵심 개념을 자연스럽게 적용할 수 있다고 판단하여 이 주제를 선택하였다.

구현 기능

- 캐릭터 생성 (이름 입력, 직업 선택: 전사·마법사·도적·팔라딘)
- 몬스터 종류별 능력치 설계 (슬라임·고블린·오크·드래곤)
- 턴제 전투 시스템 (공격·아이템 사용·도망 선택)
- 아이템 획득·사용·인벤토리 관리
- 전투 결과 처리 (승리·패배·도망)
- 게임 데이터 저장 및 불러오기 (텍스트 파일 기반 세이브/로드)

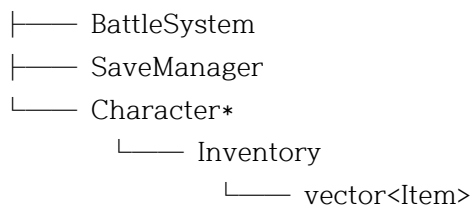
2. 클래스 설계

클래스 계층 구조는 다음과 같다.

Entity (추상 기반 클래스)



Game



Entity는 HP·공격력·방어력 같은 공통 속성과 데미지 처리를 담당한다. 캐릭터와 몬스터가 공통으로 가지는 속성이므로 하나의 기반 클래스로 묶었다.

Character는 플레이어 전용 속성인 레벨, 경험치, 인벤토리를 추가로 관리한다. 직업별 공격 방식의 차이는 Warrior·Mage·Rogue·Paladin에서 attackTarget()을 오버라이딩하여 표현하였다.

Monster는 경험치 보상과 드롭 아이템 목록을 관리한다. 몬스터 종류별 공격 방식의 차이는 각 파생 클래스에서 오버라이딩하여 표현하였다.

BattleSystem은 전투 흐름 전체를 담당한다. 플레이어 턴과 몬스터 턴을 분리하고 승패 판정 및 도망 처리를 수행한다. 전투 로직을 Game에서 분리함으로써 Game은 전체 흐름 제어에 집중할 수 있도록 하였다.

SaveManager는 파일 저장과 불러오기만 담당한다. 직업명을 기준으로 파생 클래스를 동적 생성하여 반환한다.

Inventory는 아이템 목록 관리를 담당한다. 추가·사용·제거·출력 기능을 가지며 Character 내부에 포함된다.

3. 핵심 개념 적용

Entity에 순수 가상 함수 attackTarget()과 printStatus()를 선언하였다. 이를 통해 Character와 Monster 계층 모두에서 동일한 인터페이스로 공격과 상태 출력을 처리할 수 있다. BattleSystem에서는 Character*와 Monster* 포인터로 각 객체를 다루기 때문에, 실제 타입에 따라 올바른 attackTarget()이 호출된다. 예를 들어 Warrior는 데미지 1.5배 강타, Goblin은 2회 연속 공격, Dragon은 방어력 무시 화염 공격이 각각 실행된다.

Game에서 캐릭터와 몬스터를 new로 동적 생성하고 사용 후 delete로 해제한다. Game의 소멸자에서 delete player를 호출하여 메모리 누수를 방지하였다. 또한 createMonster()에서 선택되지 않은 몬스터 객체는 즉시 delete하여 사용하지 않는 메모리를 해제한다.

예외 처리는 총 3가지 상황에 적용하였다. 전투 중 잘못된 행동 번호를 입력하면 invalid_argument를 던지고 재입력을 요청한다. 존재하지 않는 아이템 인덱스를 사용하면 out_of_range를 던진다. 세이브 파일을 열 수 없거나 알 수 없는 직업이 저장된 경우 runtime_error를 던진다.

SaveManager에서 텍스트 파일(save.txt)로 캐릭터 데이터를 저장하고 불러온다. 저장 순서는 이름, 직업명, HP, 최대HP, 공격력, 방어력, 레벨, 경험치, 인벤토리 순서이다. 불러올 때는 직업명을 기준으로 파생 클래스를 동적 생성한 뒤 저장된 스탯을 setter로 복원한다.

4. 선택 요건 적용

동적 할당은 new/delete를 사용하여 캐릭터와 몬스터를 동적으로 생성하고 해제한다. Game 소멸자에서 delete player를 호출하며, createMonster()에서 선택되지 않은 몬스터는 즉시 해제한다. 소멸자는 Entity, Character, Monster, Item, Inventory, BattleSystem, SaveManager 전 클래스에 정의되어 있다. 적용 위치는 Game.cpp의 Game::~~Game()과 Game::createMonster()이다.

STL + 알고리즘 + 람다는 vector<Monster*>로 몬스터 풀을 관리하고 vector<Item>으로 인벤토리를 관리한다. 인벤토리 출력 시 std::sort와 람다를 결합하여 아이템을 타입 순서(회복 → 공격 → 방어)로 정렬한다. 적용 위치는 Inventory.cpp의 Inventory::printInventory()이다.

5. 테스트 목록

총 5개의 단위 테스트를 test.cpp에 작성하였다.

- test_takeDamage_shouldDecreaseHp: 데미지 10, 방어력 2 조건에서 HP가 32로 감소하는지 검증하는 정상 케이스
- test_takeDamage_minDamage_shouldBeOne: 방어력이 데미지보다 클 때 최소 1 데미지가 적용되는지 검증하는 경계 케이스
- test_inventory_full_shouldReturnFalse: 인벤토리가 꽉 찼을 때 추가 시 false를 반환하는지 검증하는 실패 케이스
- test_gainExp_shouldLevelUp: 경험치 100 획득 시 레벨 2로 상승하는지 검증하는 정상 케이스
- test_setHp_shouldNotExceedMaxHp: HP 설정 시 최대 HP를 초과하지 않는지 검증하는 경계 케이스

****[실행 결과]****

단위 테스트

[PASS] 데미지_10_방어력2_HP감소

[PASS] 최소_1데미지

[PASS] 인벤토리_꽉찼을때_추가불가

테스트 레벨 업! → LV.2

[PASS] 경험치100_레벨업_LV2

[PASS] setHp_maxHp초과_안함

6. 한계 및 개선 방향

세이브 슬롯이 1개뿐이다. 현재 save.txt 하나에 덮어쓰는 방식이라 여러 캐릭터를 동시에 관리할 수 없다. 슬롯 번호를 파일명에 포함시키는 방식(save_1.txt, save_2.txt 등)으로 확장하면 멀티 슬롯을 지원할 수 있다.

몬스터 난이도가 전반적으로 낮다. 현재 $\text{expReward} \leq \text{level} * 60$ 기준이 너무 관대하여 고레벨에서도 슬라임·고블린 같은 약한 몬스터가 계속 등장할 수 있다. 또한 몬스터 스탯이 고정값이라 플레이어 레벨이 올라도 전투 난이도가 올라가지 않아 게임이 단조로워진다. 몬스터에 레벨 속성을 추가하고 스탯을 플레이어 레벨에 맞게 동적으로 스케일링하는 방향으로 개선할 수 있다.